



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA EN SISTEMAS
INGENIERIA DE SOFTWARE

Periodo

2025-B

Juegos Interactivos GR3

PROYECTO FINAL

Quito Guard: Urban Pulse

Pre-producción y Diseño Técnico

Integrantes

Johan Baño

Rocha Luis

Fecha De Entrega

31 de enero del 2026

1. Documento de Diseño de Juego (GDD) con Análisis MDA

I. Ficha Técnica y Concepto

- **Título:** Quito Guard: Urban Pulse.
- **Género:** Estrategia Táctica / Simulación de Gestión Urbana.
- **Plataforma:** Web (Babylon.js).
- **Público Objetivo:** Jugadores interesados en la gestión táctica y simulación social.
- **Elevator Pitch:** **Quito Guard** es un juego de estrategia táctica donde controlas a un coordinador de seguridad en el sector de La Mariscal. Debes desplegar patrullas comunitarias para mitigar incidentes delictivos en tiempo real, gestionando recursos limitados y la confianza ciudadana para evitar el colapso del barrio.

II. Análisis MDA (The Core)

Este análisis justifica teóricamente la experiencia del usuario desde las reglas hasta la respuesta emocional.

- **Aesthetics (Estética - La Experiencia):** Buscamos generar **Tensión** y **Responsabilidad**. El jugador debe sentir la urgencia de las alertas y el impacto de sus decisiones en la seguridad del barrio.
- **Dynamics (Dinámicas - El Comportamiento):** Emerge la **Gestión de Riesgo vs. Recompensa**. La escasez de patrullas obliga al jugador a decidir si atiende un robo menor inmediato o reserva recursos para una emergencia de alta prioridad detectada por el sistema.
- **Mechanics (Mecánicas - Las Reglas):**
 - **Sistema de Alerta:** Generación de incidentes con temporizadores basados en un JSON de configuración.
 - **Navegación de Unidades:** Movimiento con velocidad constante (\$5.0~u/s\$) y detección de colisiones mediante `moveWithCollisions`.
 - **Variable de Victoria/Derrota:** El nivel de `ConfianzaCiudadana` fluctúa según el éxito en la resolución de incidentes.

III. Mecánicas Detalladas (Game System Design)

- **Core Loop:** Monitorear Mapa $\rightarrow$ Detectar Incidente $\rightarrow$ Desplegar Patrulla $\rightarrow$ Resolver Conflicto $\rightarrow$ Ganar Confianza/Recursos.

- **Sistema de Unidades (FSM):** Las patrullas operan bajo una Máquina de Estados Finita: IDLE (En base), RESPONDING (En camino), RESOLVING (Atendiendo), RETURNING (Regresando).
- **Controles:** Mouse para selección de unidades y despliegue en el mapa; teclas de acceso rápido (1-5) para tipos de respuesta.

IV. Narrativa y Diseño de Niveles

- **Nivel 1: "La Mariscal":** Un entorno urbano denso con puntos calientes de criminalidad. El mapa incluye zonas seguras (UPC) y callejones de alto riesgo.
- **Narrativa Ambiental:** El estado de seguridad se comunica mediante la iluminación (zonas seguras cálidas, zonas de peligro con luces frías o intermitentes) y sonidos de radiofrecuencia.

2. Arquitectura Técnica (Ingeniería de Software)

I. Stack Tecnológico

- **Lenguaje:** TypeScript 5.0+ (Tipado estricto para lógica de estados).
- **Motor 3D:** Babylon.js 6.0+ (Framework robusto para WebGL).
- **Bundler:** Vite (Desarrollo local rápido).
- **Control de Versiones:** Git / GitHub.

II. Patrones de Diseño Seleccionados

- **Singleton:** Para el GameManager, asegurando una instancia única que gestione el flujo global del juego.
- **Observer:** Para comunicar el sistema de alertas con la Interfaz de Usuario (UI) sin acoplamiento directo.
- **State Pattern:** Para manejar las transiciones entre los estados de juego (Menú, Jugando, Game Over).

III. Estructura de Datos (JSON de Incidentes)

JSON

```
{
  "id": "robo_unidad",
```

```
"tipo": "Robo a Persona",  
"gravedad": "Media",  
"tiempo_resolucion": 15,  
"impacto_confianza": 5,  
"color_alerta": "#FFA500"  
}
```

3. Planificación y Gestión (Project Management)

I. Estructura de Épicas y User Stories (Backlog)

- **Épica 1: Arquitectura Base:**

- **HU:** Implementar GameManager (Singleton) para control de escenas.

- **Épica 2: Sistemas de Gameplay:**

- **HU:** Desarrollar sistema de navegación de patrullas con detección de colisiones.
- **HU:** Crear spawner de incidentes que lea desde archivos JSON.

- **Épica 3: Interfaz y Estética:**

- **HU:** Diseñar HUD reactivo para mostrar la Confianza Ciudadana y Alertas Activas.

II. Roadmap del Proyecto (Sprints de 2 semanas)

- **Sprint 1-2:** Greyboxing del mapa de Quito y configuración de arquitectura Singleton.
- **Sprint 3-4:** Implementación de la lógica de alertas y sistema de respuesta de unidades.
- **Sprint 5-6:** Integración de arte final (.glb), pulido de UI y balanceo de dificultad.

4. Post-Mortem Preventivo y Calidad

- **Riesgo Técnico:** La complejidad de la navegación en calles estrechas puede causar errores de colisión; se mitigará mediante el uso de ZoneTriggers simplificados.
- **Pruebas (QA):** Se implementará un sistema de reportes de bugs integrado para registrar errores de posicionamiento de unidades en el mapa.

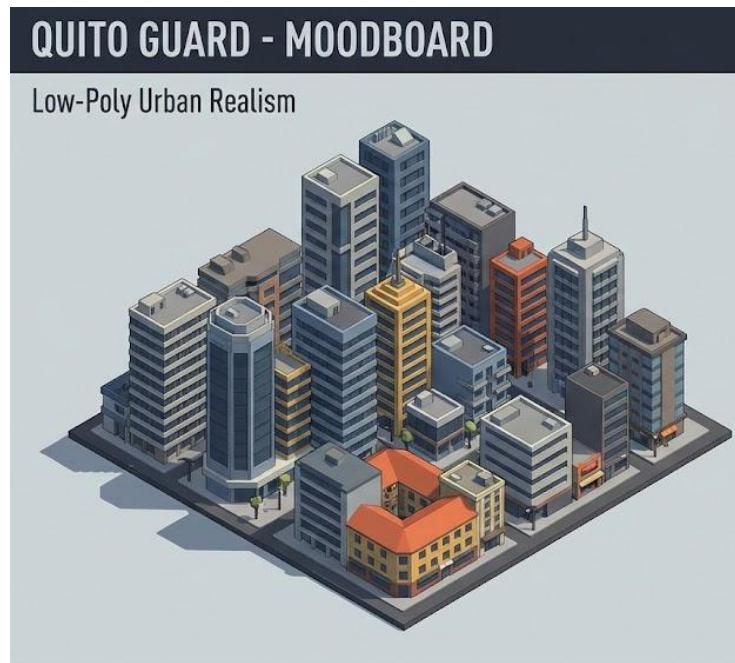
5. Changelog (Control de Versiones)

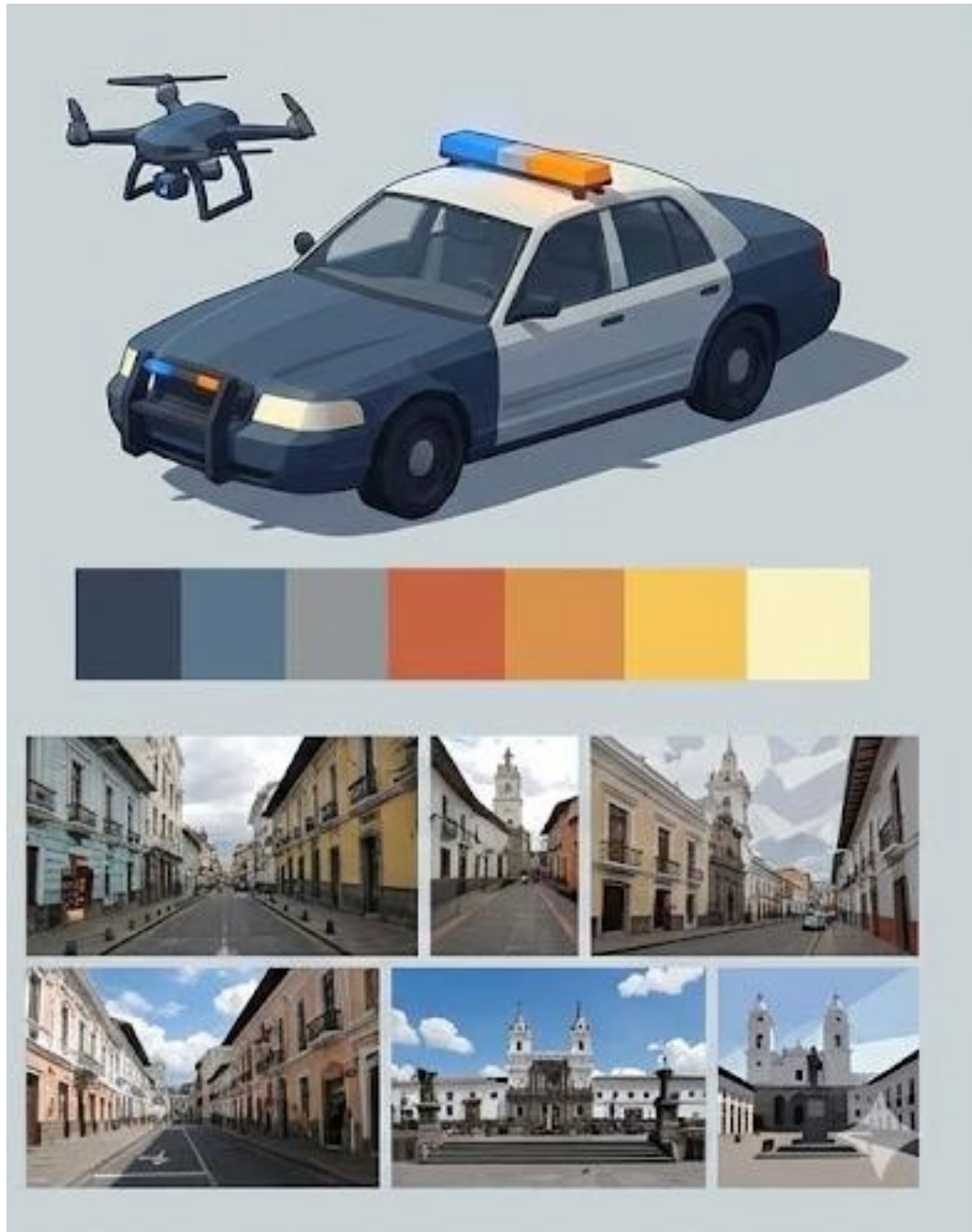
- **v1.0 (Actual):** Definición completa del GDD, Análisis MDA, Arquitectura Técnica y Planificación inicial en Backlog.

6. Anexos

1. Moodboard (Dirección de Arte)

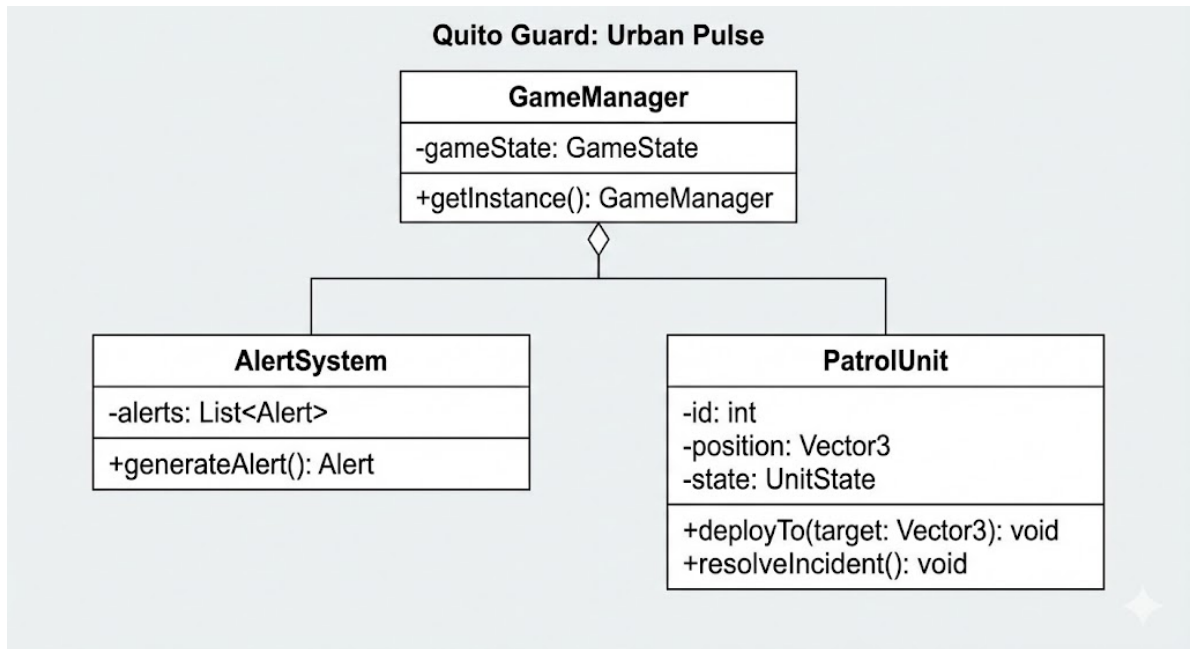
Este tablero establece la identidad visual del juego. Muestra el estilo "Low-Poly Urban Realism" con una paleta de colores sobria (azules, grises, naranjas) que transmite seriedad y tensión. Incluye referencias de la arquitectura de Quito y el diseño de una patrulla policial moderna.





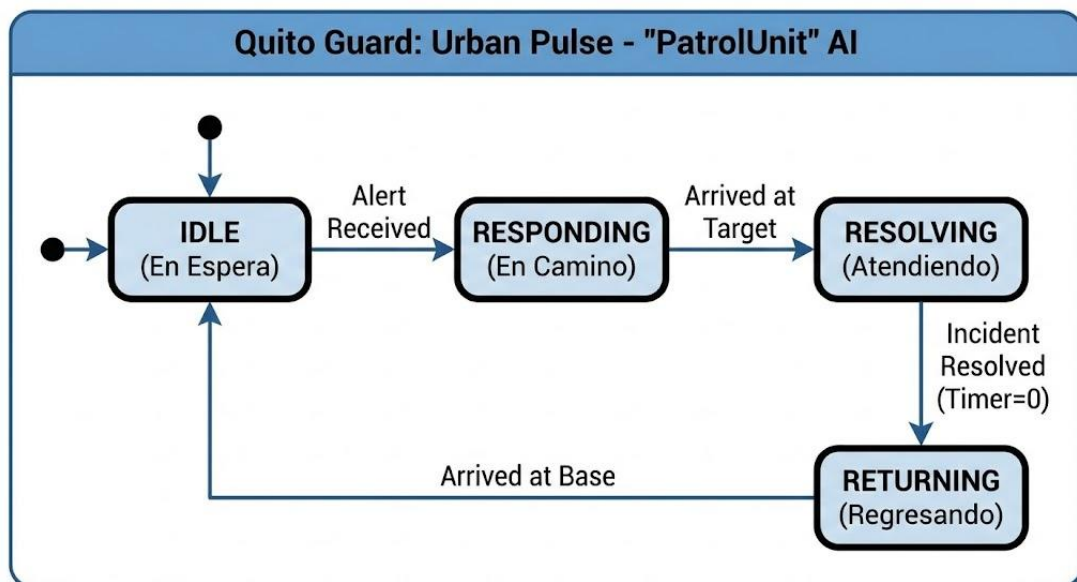
2. Diagrama de Clases UML (Arquitectura de Software)

Este es el diagrama técnico central que muestra la estructura de tu código. Ilustra cómo el GameManager (usando el patrón Singleton) actúa como el cerebro que controla el AlertSystem y las PatrolUnit. Define los atributos y métodos clave que tus programadores deberán implementar.



3. Diagrama de Máquina de Estados (Comportamiento de IA)

Este diagrama visualiza la lógica de tus patrullas. Muestra los cuatro estados que definimos (IDLE, RESPONDING, RESOLVING, RETURNING) y las condiciones exactas que provocan la transición de uno a otro. Esto es esencial para programar una IA de unidades robusta y sin errores.



4. Mapa de Greyboxing (Diseño de Niveles)

Este es el plano táctico de tu primer nivel, "La Mariscal". Muestra la disposición de las calles y edificios, identificando la base segura (UPC) y las zonas de alto riesgo donde es más probable que aparezcan incidentes. Este mapa se corresponde con los wireframes de la interfaz que ya tienes.

